



**Universidad
Nacional
de San Martín**

UNIVERSIDAD NACIONAL DE SAN MARTÍN

EXAMEN PARCIAL DE PYTHON

PROGRAMACIÓN ORIENTADA A OBJETOS, DECORADORES Y REQUESTS

Instrucciones

- Lee cuidadosamente cada consigna antes de programar.
- El examen tiene una duración de **2 horas**.
- Se evaluarán: funciones, comprensiones, uso de decoradores, requests, programación orientada a objetos y manejo de archivos y excepciones.
- Se descontarán puntos por errores de sintaxis, falta de claridad o no cumplir con el formato pedido de input y output.

1. Ejercicio 1 – Gestión de Vehículos y Decoradores (50 pts)

Se desea implementar un pequeño sistema para gestionar una flota de vehículos de una empresa de logística.

1. (Lambda y comprensiones – 8 pts) A partir de la siguiente lista de diccionarios:

```
1 vehiculos = [  
2     {"patente": "AB123CD", "tipo": "camion", "anio": 2018},  
3     {"patente": "AC555XY", "tipo": "auto", "anio": 2022},  
4     {"patente": "BB999AA", "tipo": "moto", "anio": 2019},  
5     {"patente": "CD321BA", "tipo": "camion", "anio": 2015},  
6 ]
```

Usa:

- Una función `lambda` para filtrar solo los vehículos tipo `camión`.
- Una **comprensión de listas** para obtener la lista de patentes de esos camiones.

—

2. (Clases y POO – 12 pts) Implementa una clase `Vehiculo` con:

- Atributos: `patente`, `tipo`, `anio`.
- Un método `descripcion()` que devuelva un string con el formato: `"Vehículo AB123CD - camión (2018)"`.

Luego, genera una lista de instancias de `Vehiculo` a partir de la lista original.

—

3. (Pilares de la POO – 10 pts) Extiende el sistema con las siguientes clases:

- `Auto`: imprime un mensaje indicando que tiene 5 plazas.
- `Moto`: imprime un mensaje indicando que es ágil en la ciudad.
- `Camion`: imprime un mensaje indicando capacidad de carga.

Usa herencia para evitar repetir código y sobrescribe un método `info_extra()` para demostrar polimorfismo.

—

4. (Decoradores – 10 pts) Implementa un decorador `@verificar_antiguedad` que impida registrar vehículos con más de 10 años de antigüedad. Si la antigüedad es mayor, debe lanzar una excepción personalizada `VehiculoAntiguoError`.

—

5. (Archivos y Excepciones – 10 pts) Implementa una función `guardar_vehiculos(vehiculos)` que:

- Guarde en un archivo `vehiculos.txt` cada vehículo en una línea con el formato `patente, tipo, anio`.
- Use manejo de excepciones para capturar errores de escritura y mostrar un mensaje claro en caso de fallo.

Ejemplo de ayuda: Si ocurre un error al abrir o escribir el archivo (por ejemplo, permisos denegados o ruta inexistente), tu programa debería manejarlo de forma segura, mostrando un mensaje como:

```
1 Error al guardar el archivo: [detalle del error]
```

En lugar de que el programa se interrumpa abruptamente.

2. Ejercicio 2 – Generadores y API de Ubicación de la ISS (50 pts)

Se desea construir un módulo para consultar la posición actual de la Estación Espacial Internacional (ISS) usando la API pública **Open Notify**.

1. (**Requests – 10 pts**) Realiza una solicitud HTTP GET a la siguiente URL para obtener la posición actual de la ISS:

`http://api.open-notify.org/iss-now.json`

La respuesta tendrá un formato similar a:

```
1 {
2   "message": "success",
3   "timestamp": 1710000000,
4   "iss_position": {
5     "latitude": "48.8566",
6     "longitude": "2.3522"
7   }
8 }
```

Realiza esta consulta **cinco veces** y guarda todas las respuestas en un archivo `iss.json`. Para evitar que las consultas se realicen demasiado rápido, utiliza la función `time.sleep()` entre cada una:

```
1 import time
2
3 # Ejemplo de uso:
4 for i in range(5):
5     # Hacer la petición HTTP aquí
6     time.sleep(2) # Espera 2 segundos antes de la siguiente
```

Implementa manejo de excepciones para errores de conexión (por ejemplo, si no hay Internet).

—

2. (**Archivos y Excepciones – 10 pts**) Carga el archivo `iss.json` y procesa la lista de posiciones almacenadas. Implementa control de excepciones para:

- Archivo no encontrado

Ejemplo de ayuda: Si ocurre un error al abrir el archivo, tu programa debería manejarlo de forma segura, mostrando un mensaje como:

```
1 Error al leer el archivo: [detalle del error]
```

En lugar de finalizar abruptamente.

—

3. (**Generadores – 10 pts**) Implementa un **generador** llamado `filtrar_latitud(posiciones, umbral)` que reciba la lista de posiciones y devuelva solo aquellas cuya **latitud** absoluta sea mayor al **umbral** (por ejemplo, 30). Esto permite detectar cuando la ISS se encuentra en zonas más alejadas del ecuador.

—

4. (**Lambda y comprensiones – 10 pts**) Usando:

- Una función **lambda** para convertir las coordenadas `latitude` y `longitude` de strings a **valores float redondeados** a 2 decimales.

- Una **comprensión de diccionarios** para construir un diccionario `timestamp: (lat, lon)` con las posiciones procesadas.

—

5. (Clases + Decoradores – 10 pts) Crea una clase `ReporteISS` con:

- Atributos: `fecha`, `posiciones`.
- Un método `resumen()` que imprima:
 - La latitud máxima registrada
 - La latitud mínima registrada
 - El promedio de las latitudes

Implementa un decorador `@encabezado_iss` que muestre un encabezado llamativo antes de imprimir el resumen, por ejemplo:

```
1 ===== REPORTE DE POSICIONES DE LA ISS =====
```